Story Coverage Mechanism: Implementation Design

TestAtlas Team

October 19, 2026

CONTENTS

Story Coverage Mechanism

Implementation Design: Comparing Code Against User Stories, With and Without One

1. Purpose and Scope

2. The Shared Data Model (Read This First)

3. Branch A — When a User Story Exists

3.1 Ingestion and decomposition

3.2 Candidate retrieval — before any judgment

3.3 LLM judgment — one call per requirement

3.4 Roll-up and test cross-reference

4. Branch B — When No Story Exists

4.1 Entry-point detection

4.2 Call-chain tracing per entry point

4.3 Aggregating entry points into journeys

4.4 Deriving requirements within a journey

4.5 Abstain-and-ask for what code can’t resolve

5. Reporting

6. Known Limitations (Carried Forward Honestly)

7. Build Sequencing

CONFIDENTIAL

Story Coverage Mechanism

Implementation Design: Comparing Code Against User Stories, With and Without One

---

CONFIDENTIAL

1. Purpose and Scope

This document extends the two-track design already agreed on (*Test Case Correctness & Coverage: Design Analysis*) with implementation-level detail for Track B specifically: how coverage is actually computed, mechanically, in both of its branches — when a human-written user story exists, and when it doesn't. This is a design specification, not a build plan; nothing described here has been implemented.

The governing idea carried over from that earlier document: a user story's scope is defined externally (by a person or by traced request flow), and code is checked *against* it — never the other way around. What this document adds is the actual pipeline: what gets parsed, what gets embedded, what gets sent to an LLM and with what prompt shape, what gets stored, and where the two branches converge onto one shared data model instead of staying as two parallel systems.

CONFIDENTIAL

2. The Shared Data Model (Read This First)

Both branches — story-exists and no-story — are designed to terminate in the same table shape, so that judgment, rollup, and reporting are one piece of shared logic, not two pipelines that happen to look similar.

`story_requirements` one row per atomic, checkable claim about the system, regardless of where it came from.

column	meaning
`id`	requirement id
`repo_id`	which repo
`journey_id`	nullable — set when this requirement was derived from a traced journey (Branch B) rather than a document (Branch A)
`text`	the atomic claim, e.g. “Guest users can complete checkout without an account”
`type`	`positive` or `negative` (the requirement asserts something must happen, or must not)
`source`	`document` | `derived` | `human_confirmed`
`status`	`supported` | `contradicted` | `unknown` | `needs_human_input`
`created_at`	—

`requirement_citations` — one row per grounding check against code, mirroring the citation mechanism already specified for generated test cases in the earlier document.

CONFIDENTIAL

column	meaning
`requirement_id`	FK
`node_id`	the graph node (function/file) cited as evidence
`rationale`	one-line reason the LLM judgment gave
`checked_at`	—

`journeys` — one row per candidate or confirmed user journey (Branch B only; a document-sourced story doesn't need one, since the document itself is the journey boundary).

column	meaning
`id`	journey id
`repo_id`	which repo
`name`, `description`	short, LLM-named, e.g. "Checkout flow"
`entry_point_ids`	the route/page/command nodes that make up this journey
`node_ids`	the full traced subgraph belonging to this journey

Both branches populate `story_requirements`. The branches differ entirely in **how a row gets created** (Sections 3 and 4) and are identical in **what happens once it exists** (judgment, citation, rollup — Section 3.3 onward, reused directly by Branch B via Section 4.4).

CONFIDENTIAL

3. Branch A — When a User Story Exists

3.1 Ingestion and decomposition

The story arrives the same way a process/architecture document does today, through the existing Documents feature. Decomposition into atomic requirements has two paths depending on how structured the input already is:

- **Structured input** (a bulleted or numbered acceptance-criteria list, Given/When/Then format — the common shape for a real Jira/ADO story): parsed deterministically. No LLM call. Each bullet becomes one `story_requirements` row.
- **Freeform prose**: one LLM call *per story*, not per module — this is cheap and bounded, since stories are few. Prompt: given the story text, extract a list of atomic, independently-checkable claims, each as a short imperative statement, tagged `positive` or `negative`. Output is a structured list, nothing else — this call does not judge anything against code, it only splits the document.

3.2 Candidate retrieval — before any judgment

A requirement's text is never compared against raw source directly; it's compared against a retrieval index built from text TestAtlas already generates:

- module friendly names and descriptions (already produced for the module-naming feature)
- extracted UI strings/labels (already produced as grounding input for test case generation)
- route/file paths

This index is embedded once per repo (or incrementally on re-analysis) and queried per requirement via cosine similarity, returning a short candidate list (around 8–10 nodes) rather than the whole codebase. This step is why cross-modal matching (English requirement against JS/TS code) is tractable at all: it never asks a model to read the whole repository, and it reuses an asset — the plain-English module descriptions — that already exists for an unrelated reason.

CONFIDENTIAL

Negative requirements need a different retrieval target. “Orders can’t be cancelled after dispatch” isn’t served by finding the cancellation code — it’s served by finding whether a *guard* exists near it. Initial retrieval for a negative requirement targets the positive capability it’s qualifying (here, the cancellation handler), and leaves the actual presence/absence-of-a-guard judgment to the LLM step below, with a deliberately lower expected confidence — this is flagged as a known limitation in Section 6, not solved here.

3.3 LLM judgment — one call per requirement

Given the requirement text and its candidate shortlist (real source excerpts, not descriptions), one call asks: is this implemented — cite the specific function and quote the supporting (or contradicting) line; answer `supported`, `contradicted`, or `unknown`, never a confidence number. This is the same three-state, citation-required pattern already specified for generated test case claims — reapplied here with a human requirement as the input instead of a generated step. One call per requirement keeps cost bounded and predictable: an 8-criterion story is 8 calls, not one call per module in the repository.

3.4 Roll-up and test cross-reference

Story-level percentage is a plain count: `supported / total` → *”Checkout: 75% implemented.”* Separately, for each `supported` requirement, check whether any existing test, or any already-generated test case, cites the same node (via the citation mechanism both systems share) — producing *”tested: yes/no”* per requirement, and the full 2×2 (implemented × tested) per story.

CONFIDENTIAL

4. Branch B — When No Story Exists

4.1 Entry-point detection

Framework-specific, pattern-based, deterministic — not an LLM call. This reuses the AST data TestAtlas already parses for graph construction; it's an additional pass over data already collected, not a new parser.

- Python/FastAPI-style: route decorators (`@app.get`, `@app.post`, …) detected by AST pattern match.
- Express/Node-style: `app.get(...)`, `router.post(...)` calls, same technique.
- React/Next-style: top-level route components, or — cheaper still — file-path convention (`pages/`, `app/` directory structure), which needs no AST work at all.
- CLI tools: command-registration calls (`argparse`, `click`, `commander`).

Each detected entry point is tagged on its existing graph node: `is_entry_point: true`, plus metadata (route path, HTTP method, or page path).

4.2 Call-chain tracing per entry point

Reuses the call graph already built for module clustering and PageRank. From each entry-point node, a bounded graph traversal (BFS/DFS over outbound call edges) collects the reachable subgraph — "everything that runs for this request." Nodes called by a very large number of unrelated entry points (a generic logger, a shared validator) are deprioritized by a simple call-frequency threshold, so common infrastructure doesn't get pulled into every journey.

4.3 Aggregating entry points into journeys

A single entry point is usually narrower than a real business journey (`POST /cart/add` isn't "Checkout," it's one step in it). Entry points are clustered into a candidate journey using a combined similarity score:

- overlap between their reachable subgraphs (Jaccard similarity — do they touch the same downstream code, e.g. the same `Order` model)

CONFIDENTIAL

- shared route-path prefix (`/checkout/*`)
- sequential linkage where detectable (a handler that redirects to another)

Standard hierarchical clustering over this combined score groups entry points into journeys — the same category of graph-clustering technique already used for module detection today, applied to a different input. An LLM is used only at the very end of this stage, and only for naming: given a cluster's route paths and module descriptions, produce a short name and description ("Checkout flow"). The clustering decision itself — what belongs together — stays deterministic graph math; the LLM's only job here is labeling, the same low-stakes role it already plays in module naming.

4.4 Deriving requirements within a journey

Within a journey's traced subgraph, the same category of pattern detector used for entry points looks for three shapes: validation checks (a conditional that rejects/raises), permission checks (a role/auth decorator or guard clause), and state transitions (a write to a status/state field). Each match becomes a candidate row in `story_requirements`, with `source = derived` and `journey_id` set — the same table Branch A writes to, so judgment (3.3), citation, and rollup (3.4) run unmodified against these rows too. This is the point where the two branches converge: everything from here on is shared logic.

4.5 Abstain-and-ask for what code can't resolve

A detected pattern with no way to tell if it's intended ("cancellation after dispatch is allowed — bug or feature?") is written with `status = needs_human_input` instead of a guess. These surface as a small queue in the existing review UI — the same accept/edit/reject interaction pattern already used for generated test cases, applied here to short, specific questions instead. An answered question is written back as a new row with `source = human_confirmed` — which means it now behaves exactly like a Branch-A requirement for every future analysis run on this repo. This is the concrete mechanism behind "answers compound into reusable acceptance criteria": it's a `source` field transition in one shared table, not a separate system.

CONFIDENTIAL

5. Reporting

One dashboard, reading the same `story_requirements` table regardless of branch, grouped by journey or document: requirement text, source, status, implemented %, tested %, and review status. The source column is what lets a reviewer tell at a glance which numbers rest on a human-confirmed fact and which rest on a code-derived guess still awaiting confirmation.

CONFIDENTIAL

6. Known Limitations (Carried Forward Honestly)

- **Retrieval quality bounds everything downstream.** If Section 3.2's candidate shortlist misses the right code, Section 3.3's judgment is wrong no matter how careful the prompt is. This is the standard open-ended limitation of any embedding-based retrieval system, not something solved here.
- **Negative requirements are structurally harder.** Retrieval surfaces what exists, not the absence of a violation — expect these to land in `unknown` more often than a confident verdict, by design, not by failure.
- **Entry-point and pattern detection are per-framework work.** Each new language/framework needs its own detector in Sections 4.1 and 4.4; this is real, ongoing engineering, not a one-time cost.
- **Clustering in 4.3 is heuristic.** The combined similarity score will sometimes group the wrong entry points together or split a real journey in two; this is why weak connections are shown explicitly rather than presented as settled.

CONFIDENTIAL

7. Build Sequencing

Branch A first. It reuses existing assets almost entirely (module descriptions, UI string extraction, the citation pattern) and needs no new per-framework detection work — the only new pieces are the embedding index and the per-requirement judgment call.

Branch B's shared data model, built to match Branch A from day one — even before entry-point detection exists, `story_requirements` and `requirement_citations` should already support `source = derived` and `journey_id`, so Branch B becomes new ways to populate the same table rather than a rewrite later.

Branch B's detectors last, and incrementally — one framework at a time, starting with whatever TestAtlas's own early customers actually run, rather than attempting broad framework coverage up front.

CONFIDENTIAL